

## UNIT 2

### Data Structures and Algorithms

#### LONG ANSWER TYPE QUESTIONS

##### 1. Analyze the difference between shallow copy and deep copy in Python lists.

In Python, copying lists can be done in two main ways: **shallow copy** and **deep copy**. Both are used to duplicate objects, but they behave differently when the list contains **nested or mutable elements**.

#### SHALLOW COPY

- A **shallow copy** creates a new list, but **does not create copies of nested objects** inside it.
- It **copies references** to the original objects (not the actual objects).
- Changes to nested elements in the copied list **will affect** the original list.

#### Example:

```
import copy
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
shallow[0][0] = 99
print(original)  # Output: [[99, 2], [3, 4]] – changed!
```

**Explanation:** Only the outer list is copied. Inner lists are still shared between original and shallow.

## DEEP COPY

- A **deep copy** creates a **completely independent copy**, including all nested objects.
- It duplicates everything recursively.
- Changes in the copied list **do not affect** the original.

### Example:

```
import copy
original = [[1, 2], [3, 4]]
deep = copy.deepcopy(original)
deep[0][0] = 99
print(original)  # Output: [[1, 2], [3, 4]] – unchanged!
```

### Explanation:

The `deepcopy()` function duplicates both the outer and inner lists, so they are completely separate.

## 2. Evaluate the advantages of using merge sort over bubble sort for large datasets.

Sorting is a fundamental operation in programming. Two common sorting algorithms are **Bubble Sort** and **Merge Sort**. While Bubble Sort is simple and intuitive, Merge Sort is more efficient, especially for large datasets.

### Time Complexity Comparison

|             |                          |
|-------------|--------------------------|
| Bubble Sort | Best Case : $O(n^2)$     |
| Merge Sort  | Best Case: $O(n \log n)$ |

**Merge Sort** consistently outperforms Bubble Sort, especially as the size of the dataset grows.

## Performance on Large Datasets

- **Bubble Sort** performs many redundant comparisons and swaps, making it **slow and inefficient** for large datasets.
- **Merge Sort** uses the **divide-and-conquer** approach to split the data and sort recursively, which reduces the number of operations.

### Example:

Sorting a list of 1,000 elements:

Bubble Sort may take ~1,000,000 comparisons.

Merge Sort may only need ~10,000 comparisons.

## Stability

**Merge Sort is stable**, meaning it preserves the original order of equal elements—useful in databases and multi-key sorting.

**Bubble Sort is also stable**, but its inefficiency overshadows this advantage for large data.

## Memory Usage

**Merge Sort** requires **extra memory** ( $O(n)$ ) for merging sublists.

**Bubble Sort** works **in-place** but is still too slow for most practical large-scale uses.

Merge Sort is a far better choice for large datasets due to its **predictable  $O(n \log n)$**  performance, **efficiency**, and **stability**. Bubble Sort is only suitable for educational purposes or very small datasets due to its  **$O(n^2)$**  complexity.

### 3. Develop a MATLAB script to generate a 3D array using rand() and access a specific slice.

#### WE NEED TO:

1. Generate a 3D array using `rand()`
2. Access a specific slice from it

#### MATLAB Script: Generate and Access a 3D Array

```
% Create a 3D array using rand()
% Let's create a 4x4x3 array (3 slices of 4x4 random numbers)
A = rand(4, 4, 3);

% Display the full 3D array
disp('Generated 3D array A:');
disp(A);

% Access and display the 2nd slice (i.e., along the 3rd dimension)
disp('Second slice A(:, :, 2):');
disp(A(:, :, 2));
```

#### Explanation:

`rand(4, 4, 3)`: creates a 3D array with:

**4 rows, 4 columns, 3 slices (layers)**

`A(:, :, 2)` accesses the **2nd 2D slice** in the 3rd dimension.

`disp()` is used to display matrices in MATLAB.

OUTPUT:

Generated 3D array A:

(:,: ,1) =

|        |        |        |        |
|--------|--------|--------|--------|
| 0.4018 | 0.1839 | 0.9027 | 0.3377 |
| 0.0760 | 0.2400 | 0.9448 | 0.9001 |
| 0.2399 | 0.4173 | 0.4909 | 0.3692 |
| 0.1233 | 0.0497 | 0.4893 | 0.1112 |

(:,: ,2) =

|        |        |        |        |
|--------|--------|--------|--------|
| 0.7803 | 0.0965 | 0.5752 | 0.8212 |
| 0.3897 | 0.1320 | 0.0598 | 0.0154 |
| 0.2417 | 0.9421 | 0.2348 | 0.0430 |
| 0.4039 | 0.9561 | 0.3532 | 0.1690 |

(:,: ,3) =

|        |        |        |        |
|--------|--------|--------|--------|
| 0.6491 | 0.5470 | 0.6868 | 0.7802 |
| 0.7317 | 0.2963 | 0.1835 | 0.0811 |
| 0.6477 | 0.7447 | 0.3685 | 0.9294 |
| 0.4509 | 0.1890 | 0.6256 | 0.7757 |

Second slice A(:,: , 2):

|        |        |        |        |
|--------|--------|--------|--------|
| 0.7803 | 0.0965 | 0.5752 | 0.8212 |
| 0.3897 | 0.1320 | 0.0598 | 0.0154 |
| 0.2417 | 0.9421 | 0.2348 | 0.0430 |
| 0.4039 | 0.9561 | 0.3532 | 0.1690 |

## UNIT 3

### Numerical Methods in Scientific Programming:

#### Short Answer Type Questions

##### 1. Define truncation error.

Truncation error happens when we cut off or stop a long or infinite process and use only part of it.

For example:

We stop an infinite series after a few terms.

Or we estimate a curve using straight lines.

This stopping introduces a small mistake — that's the truncation error.

##### 2. What is the difference between forward and central difference methods?

**Forward Difference:** Uses the current point and the next one to estimate a derivative.

Example: Uses  $f(x)$  and  $f(x+h)$

**Central Difference:** Uses the point before and after the current one.

Example: Uses  $f(x-h)$  and  $f(x+h)$

**Central difference is more accurate** because the error is smaller (proportional to  $h^2$ ).

##### 3. Under what conditions does Simpson's rule provide better results than the trapezoidal rule?

**Simpson's Rule** is better when the function is smooth (no sharp corners or jumps).

It fits **curved (parabolic)** shapes better.

Trapezoidal Rule connects points with **straight lines**, so it's not as accurate for curved graphs.

So, use Simpson's rule for smooth, curved functions.

#### 4. When would you prefer Gauss-Seidel over Gaussian elimination?

**Gauss-Seidel** is better for **large systems** of equations, especially if there are many zeros in the matrix.

It's **faster and uses less memory** because it solves step by step (iterative).

**Gaussian Elimination** is more exact but uses more memory and time for large systems.

Use Gauss-Seidel when the system is big and the matrix has many zero values.

#### 5. Which method would you choose for a high-dimensional integral – Simpson's or Monte Carlo – and why?

**Simpson's Rule** becomes very slow and hard to use when you have many variables (high dimensions).

**Monte Carlo Method** is better because it uses **random sampling**, and it works well even in high dimensions.

For integrals with many variables, always go with **Monte Carlo** – it's more practical.

## Long Answer Type Questions

### 1. Evaluate the accuracy of Trapezoidal and Simpson's Rule by comparing their error terms.

#### Accuracy Evaluation:

The accuracy of numerical integration methods is often assessed by their error terms, which describe how the error decreases as the step size ( $h$ ) or number of subintervals ( $n$ ) changes.

#### Trapezoidal Rule Error Term:

The global error for the Trapezoidal Rule for a single interval  $[a, b]$  with step size  $h = b - a$  is given by:

$$E_T = -\frac{(b-a)^3}{12} f''(\xi)$$

This shows that the Trapezoidal Rule is a second-order method ( $O(h^2)$ ). Doubling the number of intervals (halving  $h$ ) reduces the error by a factor of four.

#### Simpson's Rule Error Term:

The global error for Simpson's Rule for a single pair of intervals (i.e., over  $2h$ ) is given by:

$$E_S = -\frac{(b-a)^5}{2880} f^{(4)}(\xi)$$

This shows that Simpson's Rule is a fourth-order method ( $O(h^4)$ ). Doubling the number of intervals (halving  $h$ ) reduces the error by a factor of sixteen.

#### Comparison:

Simpson's Rule is significantly more accurate than the Trapezoidal Rule for smooth functions. Its error term depends on the fourth derivative of the function and is proportional to  $h^4$ , whereas the Trapezoidal Rule's error depends on the second derivative and is proportional to  $h^2$ . This means that for the same step size  $h$ , Simpson's Rule will typically yield a much smaller error. If the function is a polynomial of degree three or less, Simpson's Rule will yield the exact integral, as its fourth derivative would be zero.



## **2. Assess the trade-offs between direct and iterative methods for solving linear systems.**

**FYI:trade-offs means comparing advantages(pros) and disadvantages(cons) to make a decision.**

### **Direct Methods (e.g., Gaussian Elimination, LU Decomposition):**

#### **Pros:**

They guarantee an exact solution (up to round-off errors) if the system has a unique solution.

The computation time is fixed and predictable.

They are generally more reliable for ill-conditioned systems.

#### **Cons:**

They are computationally expensive for large systems due to high time complexity ( $O(n^3)$ ).

They require a lot of memory as they store the entire matrix, including zeros.

They are prone to round-off errors, especially for large or ill-conditioned systems.

### **Iterative Methods (e.g., Jacobi, Gauss-Seidel, Conjugate Gradient):**

#### **Pros:**

They use less memory by storing only non-zero elements, making them suitable for sparse systems.

They are often faster than direct methods for large, sparse problems.

They are less affected by round-off errors because each step builds on the previous one.

They can be stopped early when an approximate solution is good enough.

#### **Cons:**

They do not always guarantee convergence unless the system meets certain conditions.

They may converge slowly, especially for difficult systems.

They need a stopping rule to decide when the solution is accurate enough.

```
import numpy as np
from scipy.linalg import solve
from scipy.sparse.linalg import cg

A = np.array([[4, 1], [1, 3]])
b = np.array([1, 2])

# Direct Method
x_direct = np.linalg.solve(A, b)

# Iterative (Conjugate Gradient)
x_iterative, info = cg(A, b)

print("Direct solution:", x_direct)
print("Iterative (CG) solution:", x_iterative)
output:
Direct solution: [0.09090909 0.63636364]
Iterative (CG) solution: [0.09090909 0.63636364]
```

### 3. Evaluate the limitations of numerical differentiation methods and suggest improvements.

#### Limitations of Numerical Differentiation Methods:

##### Sensitive to Noise and Round-off Errors:

- Subtracting nearly equal numbers (especially with small step size) can cause big errors, especially with noisy data.

##### Step Size Trade-off:

- Small  $h$  reduces formula error but increases round-off error.
  - Large  $h$  reduces round-off error but gives less accurate results.
- Finding the best  $h$  is difficult.

##### Low Accuracy:

- Basic methods like forward/backward differences are not very accurate unless  $h$  is very small.

### **Boundary Problems:**

- Central difference needs values on both sides; at edges, less accurate one-sided formulas are used.

### **Needs Smooth Functions:**

- If the function has sharp corners or jumps, the method gives poor results.

### **Suggested Improvements:**

#### **Use Higher-Order Formulas:**

- More accurate formulas like the five-point central difference allow better results with larger  $h$ .

#### **Use Symbolic Differentiation:**

- If the formula is known, tools like SymPy give exact derivatives without errors.

#### **Use Richardson Extrapolation:**

- Combine results with different step sizes to improve accuracy.

#### **Smooth Before Differentiating:**

- Fit a smooth spline (like a cubic spline) and then differentiate.

Use filters like Savitzky-Golay to smooth and differentiate noisy data.

#### **Use Complex Step Method:**

- If possible, evaluate the function at  $x + ih$  to get very accurate derivatives without round-off errors.

#### 4. Create a Python script that compares Trapezoidal, Simpson's, and Monte Carlo integration methods.

```
import numpy as np

# Common Setup
a = 0
b = np.pi
n = 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)
f = np.sin(x)

# --- Trapezoidal Rule ---
I_trap = (h/2) * (f[0] + 2 * sum(f[1:n-1]) + f[n-1])
err_trap = 2 - I_trap

print("Trap =", I_trap)
print("Trap_Error =", err_trap)

# --- Simpson's Rule ---
I_simp = (h/3) * (f[0] + 2 * sum(f[2:n-2:2]) + 4 * sum(f[1:n-1:2]) + f[n-1])
err_simp = 2 - I_simp

print("Simp =", I_simp)
print("Simp_Error =", err_simp)

# --- Monte Carlo Integration ---
N = 10000 # number of random samples
rand_x = np.random.uniform(a, b, N)
f_rand = np.sin(rand_x)
I_mc = (b - a) * np.mean(f_rand)
err_mc = 2 - I_mc
print("Monte =", I_mc)
print("Monte_Error =", err_mc)

OUTPUT:
Trap = 1.9835235375094546
Trap_Error = 0.01647646249054535
Simp = 2.0001095173150043
Simp_Error = -0.00010951731500430384
Monte = 2.0074082916249822
Monte_Error = -0.007408291624982244
```

**5. Design a MATLAB tool that numerically differentiates a function using forward, backward, and central difference methods with graphical output.**

```
% Define function and domain
f = @(x) sin(x);
h = 0.1;
x = 0:h:2*pi;

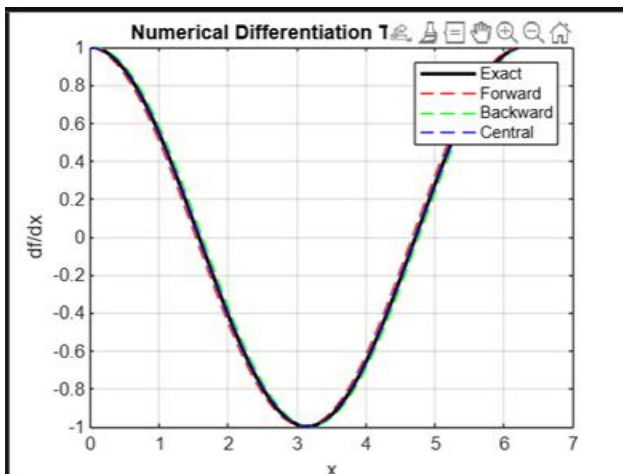
% Forward Difference
fwd = (f(x + h) - f(x)) ./ h;

% Backward Difference
bwd = (f(x) - f(x - h)) ./ h;

% Central Difference
cen = (f(x + h) - f(x - h)) ./ (2 * h);

% Exact Derivative
true = cos(x);

% Plotting
plot(x, true, 'k-', 'LineWidth', 2); hold on;
plot(x, fwd, '--r');
plot(x, bwd, '--g');
plot(x, cen, '--b');
legend('Exact', 'Forward', 'Backward', 'Central');
xlabel('x'); ylabel('df/dx');
title('Numerical Differentiation Tool in MATLAB');
grid on;
```



- The MATLAB code numerically differentiates the function  $\sin(x)$  using forward, backward, and central difference methods and visually compares them to the exact derivative  $\cos(x)$ .
- It begins by defining the function, step size  $h$ , and generating a range of  $x$  values from 0 to  $2\pi$ . It then computes the approximate derivatives using each method:
- forward difference uses the next point, backward difference uses the previous point, and central difference uses both to improve accuracy.
- The exact derivative is also computed for comparison.
- Finally, the code plots all four results—exact, forward, backward, and central—on the same graph with appropriate labels, colors, and a legend, making it a useful tool for visually understanding and comparing the accuracy of numerical differentiation methods.

## Unit 4

### Data Analysis and Visualization

#### Short Answer Type Questions

**1. Suggest a suitable visualization dashboard for a hospital's patient data monitoring system.**

A suitable visualization dashboard for a hospital's patient data monitoring system should include real-time line graphs for vital signs (like heart rate, temperature, and oxygen levels), pie charts for patient category distribution (ICU, OPD, emergency), bar charts for daily admissions/discharges, and alert indicators for critical cases. Tools like **Plotly Dash**, **Tableau**, or **Power BI** can offer interactive features to track patient health and hospital resource usage efficiently.

**2. How would you design a visualization to represent customer feedback data across different regions?**

To visualize customer feedback across regions, use a **heatmap** to represent regional sentiment scores. Combine it with **bar charts** showing counts of positive, neutral, and negative feedback per region. For deeper insight, **word clouds** or **box plots** of text sentiment scores can also be used. Tools like **Seaborn**, **Plotly**, or **Power BI** allow these visualizations to be interactive and regionally filterable.

## Long Answer Type Questions

### 1. Write a Python program using Matplotlib to draw a customized bar chart for —Total Bill vs. Day from the tips.csv dataset.

```
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
tips = pd.read_csv('tips.csv')
# Group by day and calculate total bill
grouped = tips.groupby('day')['total_bill'].sum().reset_index()

# Plotting
plt.figure(figsize=(8, 5))
plt.bar(grouped['day'], grouped['total_bill'], color='teal',
        edgecolor='black')
plt.title('Total Bill vs Day')
plt.xlabel('Day')
plt.ylabel('Total Bill ($)')
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

This program loads the **tips.csv** dataset, groups data by day, and sums the total\_bill for each. Then it draws a customized bar chart using Matplotlib with a clear title, labels, colors, and grid lines to enhance readability.

### 2. Assess the effectiveness of interactive dashboards in communicating insights to non-technical stakeholders.

Interactive dashboards are powerful tools for helping non-technical stakeholders understand complex data. They use visuals like charts, graphs, and colors to make patterns and trends easier to recognize, allowing users to grasp insights quickly without needing technical skills.



## **ADVANTAGES:**

### **Better Understanding through Visuals:**

Dashboards use charts and colors to highlight trends and outliers, making data easier to understand than plain numbers.

### **Empowers Users:**

Non-technical users can explore data themselves by filtering and drilling down, leading to quicker and more informed decisions.

### **Greater Engagement:**

Dashboards can tell a story and make data feel more relevant, increasing trust and buy-in from users.

### **Real-Time Updates:**

Live data ensures users are always seeing the most up-to-date information, which is helpful for decision-making.

## **CHALLENGES:**

### **Too Much Information:**

Cluttered dashboards with too many elements can confuse users.

### **Misleading Visuals:**

Wrong chart types or missing context can lead to incorrect conclusions.

### **Complex Interactions:**

If the dashboard is hard to navigate, non-technical users may get frustrated.

### **Oversimplification:**

Important details may be lost if visuals are too simplified.

## Cost and Maintenance:

Building and maintaining dashboards with real-time data can be expensive and require technical expertise.

## Design Effective Dashboard :

- Keep it **simple and clear**.
- Focus on the **user's needs** and key KPIs.
- Use **interactive elements** that are easy to use.
- Provide **context** (benchmarks, trends).
- Ensure the dashboard is **fast** and responsive.
- Offer **basic training or help guides** if needed.

## 3. Design a comparative dashboard using Matplotlib and Seaborn for sales data showing monthly trends, category performance, and regional distribution.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Sample sales data (replace with your actual dataset)
data = pd.DataFrame({
    'Month': ['Jan', 'Feb', 'Mar', 'Apr']*3,
    'Sales': [100, 120, 150, 130, 90, 95, 100, 110, 200, 210, 190, 220],
    'Category': ['Electronics']*4 + ['Clothing']*4 + ['Furniture']*4,
    'Region': ['North', 'South', 'East', 'West']*3
})

# Set up the dashboard layout
plt.figure(figsize=(15, 5))

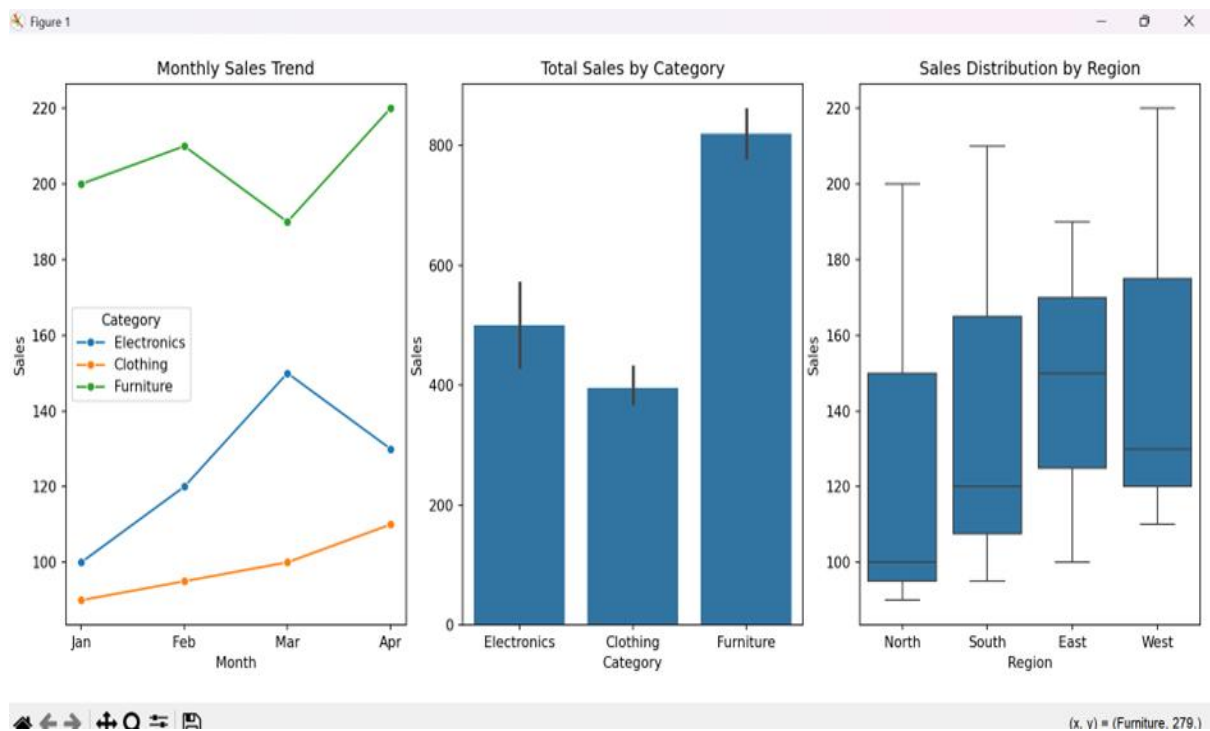
# Monthly trend
plt.subplot(1, 3, 1)
sns.lineplot(data=data, x='Month', y='Sales', hue='Category', marker='o')
```

```
plt.title('Monthly Sales Trend')

# Category performance
plt.subplot(1, 3, 2)
sns.barplot(data=data, x='Category', y='Sales', estimator=sum)
plt.title('Total Sales by Category')

# Regional distribution
plt.subplot(1, 3, 3)
sns.boxplot(data=data, x='Region', y='Sales')
plt.title('Sales Distribution by Region')

plt.tight_layout()
plt.show()
```



## Unit-5

### Emerging Technologies and Project Work

#### Short Answer Type Questions

##### 1. Differentiate between Edge computing and Cloud computing in IoT.

###### EDGE COMPUTING

Edge computing means processing data **closer to where it is generated** (i.e., at the "edge" of the network), instead of sending it all to a distant cloud server.

###### Example:

A self-driving car processes data from its sensors **immediately** on-board to make split-second decisions. It **can't wait** to send data to a cloud and receive instructions.

###### CLOUD COMPUTING

Cloud computing means sending data to a **centralized server (the cloud)** over the internet for processing, storing, and managing applications or services.

###### Example:

Google Drive or Dropbox stores your files on cloud servers. You can access them anytime from any device using the internet.

## **Long Answer Type Questions**

### **1. Explain the components and architecture layers of IoT.**

#### **Components of IoT system:**

##### **Things/Devices: Sensors, actuators.**

These are the physical objects like sensors (to collect data such as temperature or motion) and actuators (to perform actions like turning on a light or motor).

##### **Connectivity: Networks (Wi-Fi, LTE).**

This connects the devices to the internet or each other using Wi-Fi, Bluetooth, LTE, or Zigbee so they can send and receive data.

##### **Data Processing Units: Microcontrollers, cloud servers.**

These are microcontrollers (e.g., Arduino, Raspberry Pi) or cloud servers that process the collected data and make decisions (e.g., trigger an alert or start a machine).

##### **User Interface: Mobile or web apps to visualize data and send commands.**

This is how users interact with the system—using mobile apps or web dashboards to view data, receive alerts, or control devices remotely.

### **IOT ARCHITECTURE**

IoT architecture consists of **four key layers**:

#### **Perception Layer (Sensing Layer):**

Includes sensors and devices that collect physical data like temperature, motion, etc.

Example: RFID tags, cameras, GPS, and IoT sensors.

#### **Network Layer (Transmission Layer):**

Responsible for transmitting data collected by sensors to processing units.

Technologies: Wi-Fi, Bluetooth, 5G, Zigbee.

### **Processing Layer (Middleware):**

Performs data analysis, filtering, and processing.

Involves edge computing or cloud services.

### **Application Layer:**

Delivers specific services to users based on processed data.

Examples: Smart homes, smart healthcare, smart agriculture.

## **COMPONENTS OF IOT SYSTEM:**

### **Things/Devices: Sensors, actuators.**

These are the physical objects like sensors (to collect data such as temperature or motion) and actuators (to perform actions like turning on a light or motor).

### **Connectivity: Networks (Wi-Fi, LTE).**

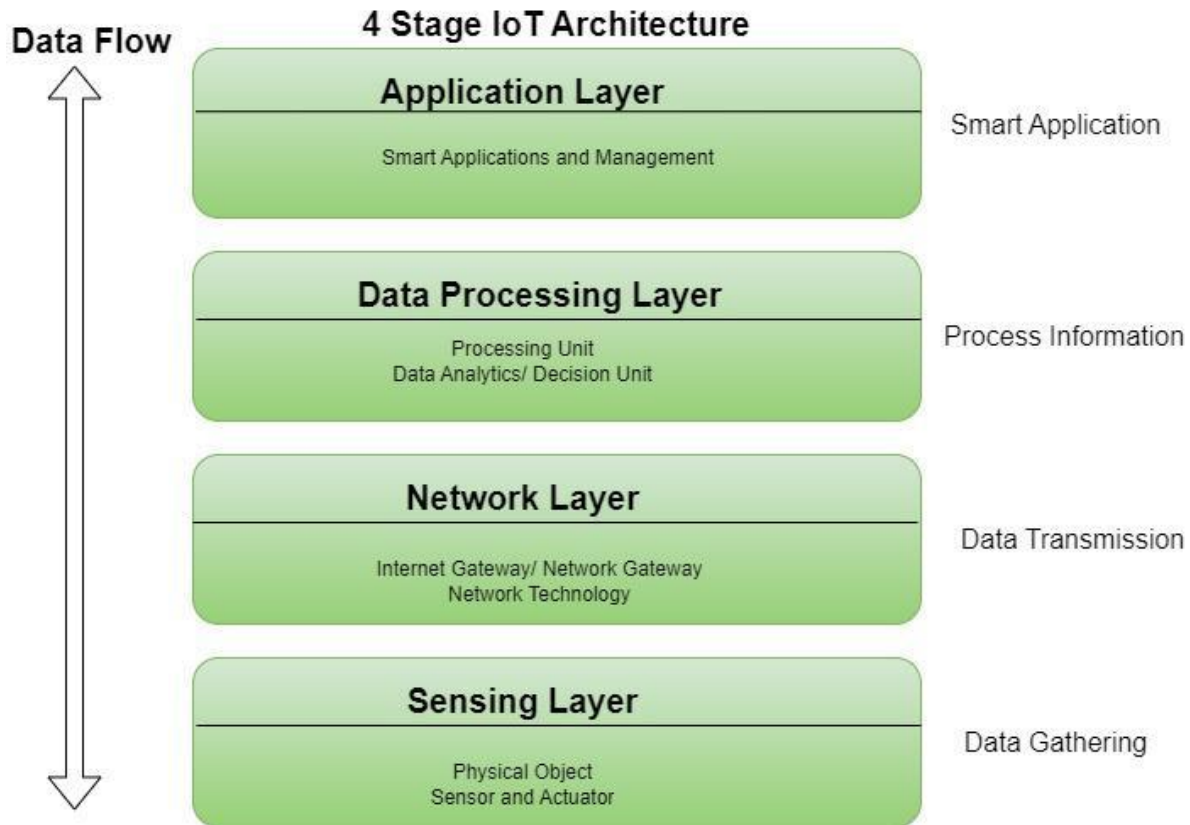
This connects the devices to the internet or each other using Wi-Fi, Bluetooth, LTE, or Zigbee so they can send and receive data.

### **Data Processing Units: Microcontrollers, cloud servers.**

These are microcontrollers (e.g., Arduino, Raspberry Pi) or cloud servers that process the collected data and make decisions (e.g., trigger an alert or start a machine).

### **User Interface: Mobile or web apps to visualize data and send commands.**

This is how users interact with the system—using mobile apps or web dashboards to view data, receive alerts, or control devices remotely.



## 2. Apply ML and IoT to design a smart irrigation system.

A smart irrigation system uses ML and IoT to water crops efficiently, saving water and improving yields.

### 1. IOT FOR DATA COLLECTION:

**Sensors:** Gather real-time data like soil moisture, temperature, humidity, and rainfall.

**Microcontrollers (e.g., ESP32, Arduino):** Collect sensor data and send it to the cloud.

**Actuators (e.g., Solenoid Valves, Pumps):** Control water flow based on system decisions.

## 2. ML FOR SMART DECISIONS:

**Data Analysis:** ML models (e.g., Regression, Classification) analyze sensor data, historical weather, and crop needs.

**Prediction:** They predict exactly when and how much water each crop zone requires.

**Automation:** Based on predictions, ML triggers the actuators to irrigate automatically.

## 3. HOW IT WORKS (Simple Flow):

**Sense:** IoT sensors collect field data.

**Send:** Data goes to the cloud.

**Analyze:** ML processes data and decides irrigation needs.

**Act:** Commands sent to actuators to water only when necessary.

**Monitor:** Farmers see data and control via app/web.

## BENEFITS:

**Water Saving:** Only irrigates when needed.

**Better Yield:** Provides optimal water for healthy crops.

**Less Labor:** Automates irrigation tasks.

**Remote Control:** Manage fields from anywhere.

## 3. Examine how Big Data analytics helps in supply chain optimization.

Big Data analytics helps improve decision-making in supply chain management by analyzing large volumes of data from various sources.

### Demand Forecasting:

By studying past sales data, Big Data can predict what products will be needed in the future. This helps companies stock the right items at the right time.



### **Inventory Optimization:**

Real-time tracking of stock levels prevents both overstocking (wasting money and space) and understocking (losing sales).

### **Route Optimization:**

Big Data uses GPS and traffic data to find the fastest, most fuel-efficient routes, reducing delivery times and transport costs.

### **Supplier Performance Monitoring:**

It tracks delivery timelines and product quality to help companies choose the most reliable suppliers.

### **Risk Management:**

Big Data can detect potential disruptions like weather or political issues and suggest changes to minimize their impact.

**Therefore** ,Big Data makes supply chains more **efficient, flexible, and smart**, helping businesses respond quickly to changes and make better decisions.

## **4. Assess the ethical issues in using IoT for personal finance management in banking.**

### **Ethical Issues in Using IoT for Personal Finance Management in Banking**

#### **Data Privacy:**

Financial data from IoT devices (e.g., wearables, smart cards) may be collected without clear user consent.

### **Data Security:**

IoT devices are prone to hacking, which can expose sensitive personal and banking information.

### **Informed Consent:**

Users often do not fully understand what data is being collected or how it will be used.

### **Algorithmic Bias:**

Machine learning models using IoT data may make biased or unfair decisions (e.g., in credit scoring or loan approval).

### **Surveillance and Control:**

Continuous monitoring of user spending habits may lead to discrimination or a loss of personal freedom.

### **Lack of Transparency:**

Companies may not clearly disclose how IoT-collected data is used, stored, or shared.

**Therefore,** While IoT improves banking services, strong data protection laws, user awareness, and ethical system design are essential to maintain trust and protect rights.